



UTT

UNIVERSIDAD TECNOLÓGICA DE TIJUANA

GOBIERNO DE BAJA CALIFORNIA

TEMA

Selection of Design Patterns topic

PRESENTADO POR

Lopez Domínguez Luis Enrique

GRUPO

10° B

MATERIA

Desarrollo móvil integral

PROFESOR

Ray Parra

Tijuana, Baja California, 07 de enero del 2025

Instructions:

1. Search different sources of information for the Selection of Design Patterns topic
2. Read and Understand the topic
3. Make a summary of the topic
4. Convert the summary to PDF
5. Upload the PDF to the corresponding assignment
6. Wait for feedback from the teacher

Selection of Design Patterns topic

Strategy

Es un patrón de diseño de comportamiento que te permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.

Aplicabilidad

1. Utiliza el patrón Strategy cuando quieras utilizar distintas variantes de un algoritmo dentro de un objeto y poder cambiar de un algoritmo a otro durante el tiempo de ejecución.
2. El patrón Strategy te permite alterar indirectamente el comportamiento del objeto durante el tiempo de ejecución asociándolo con distintos subobjetos que pueden realizar subtarefas específicas de distintas maneras.
3. Utiliza el patrón Strategy cuando tengas muchas clases similares que sólo se diferencien en la forma en que ejecutan cierto comportamiento.
4. El patrón Strategy te permite extraer el comportamiento variante para ponerlo en una jerarquía de clases separada y combinar las clases originales en una, reduciendo con ello el código duplicado.
5. Utiliza el patrón para aislar la lógica de negocio de una clase, de los detalles de implementación de algoritmos que pueden no ser tan importantes en el contexto de esa lógica.
6. El patrón Strategy te permite aislar el código, los datos internos y las dependencias de varios algoritmos, del resto del código. Los diversos clientes obtienen una interfaz simple para ejecutar los algoritmos y cambiarlos durante el tiempo de ejecución.

Pros y contras

- | | |
|--|---|
| ✓ Puedes intercambiar algoritmos usados dentro de un objeto durante el tiempo de ejecución. | ✗ Si sólo tienes un par de algoritmos que raramente cambian, no hay una razón real para complicar el programa en exceso con nuevas clases e interfaces que vengan con el patrón. |
| ✓ Puedes aislar los detalles de implementación de un algoritmo del código que lo utiliza. | ✗ Los clientes deben conocer las diferencias entre estrategias para poder seleccionar la adecuada. |
| ✓ Puedes sustituir la herencia por composición. | ✗ Muchos lenguajes de programación modernos tienen un soporte de tipo funcional que te permite implementar distintas versiones de un algoritmo dentro de un grupo de funciones anónimas. Entonces puedes utilizar estas funciones exactamente como habrías utilizado los objetos de estrategia, pero sin saturar tu código con clases e interfaces adicionales. |
| ✓ <i>Principio de abierto/cerrado.</i> Puedes introducir nuevas estrategias sin tener que cambiar el contexto. | |

Decorator

Es un patrón de diseño estructural que te permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funcionalidades.

Aplicabilidad

1. Utiliza el patrón Decorator cuando necesites asignar funcionalidades adicionales a objetos durante el tiempo de ejecución sin descomponer el código que utiliza esos objetos.
2. El patrón Decorator te permite estructurar tu lógica de negocio en capas, crear un decorador para cada capa y componer objetos con varias combinaciones de esta lógica, durante el tiempo de ejecución. El código cliente puede tratar a todos estos objetos de la misma forma, ya que todos siguen una interfaz común.
3. Utiliza el patrón cuando resulte extraño o no sea posible extender el comportamiento de un objeto utilizando la herencia.
4. Muchos lenguajes de programación cuentan con la palabra clave final que puede utilizarse para evitar que una clase siga extendiéndose. Para una clase final, la única forma de reutilizar el comportamiento existente será envolver la clase con tu propio wrapper, utilizando el patrón Decorator.

Pros y contras

- | | |
|--|--|
| ✓ Puedes extender el comportamiento de un objeto sin crear una nueva subclase. | ✗ Resulta difícil eliminar un wrapper específico de la pila de wrappers. |
| ✓ Puedes añadir o eliminar responsabilidades de un objeto durante el tiempo de ejecución. | ✗ Es difícil implementar un decorador de tal forma que su comportamiento no dependa del orden en la pila de decoradores. |
| ✓ Puedes combinar varios comportamientos envolviendo un objeto con varios decoradores. | ✗ El código de configuración inicial de las capas pueden tener un aspecto desagradable. |
| ✓ <i>Principio de responsabilidad única.</i>
Puedes dividir una clase monolítica que implementa muchas variantes posibles de comportamiento, en varias clases más pequeñas. | |

Justificación

No tengo experiencia trabajando con patrones de diseño, pero elegí Strategy y Decorator porque son complementarios y útiles. Strategy me resulta similar a la programación orientada a objetos, ya que encapsula algoritmos y permite cambiarlos dinámicamente, lo que lo hace cómodo para mí. Por otro lado, Decorator es ampliamente usado en frameworks como NestJS, y estoy interesado en aprenderlo porque permite extender funcionalidades de manera flexible sin modificar la estructura original del objeto. Ambos patrones me parecen prácticos y relevantes para resolver problemas de diseño modernos.